Semaphore-Bounded Concurrency, Worked Out by Hand

Why a semaphore of size k turns N slow fetches into $\lceil N/k \rceil$ waves —
and why the speedup is linear in k right up until the provider says no.

What this document does. It takes the parallel stages of the Module 2.3 research engine — `FETCH` and `MAP`, both gated by an `asyncio.Semaphore` so that at most `max_concurrency` coroutines run at once — and computes, by hand, exactly how long the work takes as a function of that limit. We fix one tiny example: $N = 20$ fetches, each lasting $d = 0.5$ seconds, run under a semaphore of size k. From those numbers we derive the wall-clock time, the throughput, the peak concurrency, and the speedup over serial, for $k = 1, 2, 5, 10, 20$. Every figure is reproduced by the small Python program in Appendix A. The structure is the module's own: its `map_sources` runs `abatch` with `max_concurrency=12` over 24 sources — the same $\lceil N/k \rceil$ wave arithmetic we work below.

Where this sits. This is **Topic 2 of Module 2.3** (Research Summarization Engine) — the timing model behind its "async + gated parallelism" constraint and the `sem` in `fetch_sources`. Its companions: Module 2.2 (map-reduce, where the gated `abatch` pattern is introduced) and Module 0.0 (the cost model — concurrency buys you wall-time, never tokens).

Concepts covered, in order: N tasks of duration d under a semaphore of size k · the wave count $\lceil N/k \rceil$ · wall time $\approx \lceil N/k \rceil\, d$ · throughput k/d · Little's law $L = \text{throughput} \times d = k$ · speedup linear in k until $k \geq N$ · why rate limits and memory cap k below N · the wall-time / peak-concurrency tradeoff table.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: many fetches, one semaphore

The research engine fans out. `FETCH` fires a retrieval coroutine per sub-question; `MAP` fires a summarization coroutine per source. If you launched all of them at once you would flood the provider — so both stages wrap the work in an `asyncio.Semaphore(k)`: a coroutine must *acquire* one of k permits before it runs and *releases* it when done. At any instant, at most k tasks are in flight. That is the whole mechanism.

We model the cost of that mechanism. Suppose there are N tasks, each taking the *same* duration d (seconds of mostly-waiting-on-the-network), and the semaphore holds k permits. The tasks are independent; the only thing serialising them is the permit count.

Quantity	Symbol	Value here
Number of tasks (e.g. fetches)	N	20
Duration of one task	d	0.5 s
Semaphore size / concurrency limit	k	$1, 2, 5, 10, 20$
Number of waves	$W = \lceil N/k \rceil$	derived
Wall-clock time	T	derived
Throughput	λ	derived

Notation. $\lceil x \rceil$ is the ceiling — the smallest integer $\geq x$. A *wave* is one batch of up to k tasks running together; because all tasks share the duration d, a wave finishes in d seconds and the next wave begins. Throughput λ is tasks completed per second once the pipeline is full. We assume tasks are network-bound, so k of them truly overlap rather than fighting for one CPU.

Intuition

A semaphore is a bouncer with k wristbands. N people want in; only k can be inside at once. Everyone stays for the same time d, so the room empties and refills in lockstep: k in, wait d, k out, next k in. The line clears in $\lceil N/k \rceil$ such cycles. Raising k hands out more wristbands and clears the line faster — but the room (the provider's rate limit, your RAM) only holds so many bodies before it breaks. The whole design question is: how big can k get before the room complains?

2 Step 1 — counting waves

Tasks run k at a time. The number of waves needed to clear N tasks is

$$W = \left\lceil \frac{N}{k} \right\rceil, \tag{1}$$

the ceiling because a final partial group of fewer than k tasks still costs one whole wave of time. For $N = 20$, $k = 5$: $W = \lceil 20/5 \rceil = \lceil 4 \rceil = 4$ waves. For $k = 10$: $W = \lceil 2 \rceil = 2$. For $k = 20$ (or anything $\geq N$): $W = 1$ — one wave holds everyone, and handing out more wristbands than there are people buys nothing.

Mini-example — this operation in isolation

The ceiling is the part people forget. With $N = 20$, $k = 6$ you might guess "about 3 waves". But $20/6 = 3.33$, and you cannot run a third-of-a-wave: $\lceil 3.33 \rceil = 4$. The leftover $20 - 3 \cdot 6 = 2$ tasks occupy a fourth wave almost to themselves, wasting most of that wave's capacity. Wave counts jump in integer steps; picking k to *divide* N evenly (here $k \in \{1, 2, 4, 5, 10, 20\}$) keeps every wave full.

3 Step 2 — wall time, throughput, and Little's law

Each wave takes d seconds, and there are W of them, so the wall-clock time is

$$T \approx W\, d = \left\lceil \frac{N}{k} \right\rceil d. \tag{2}$$

For our run, $N = 20$, $d = 0.5$, $k = 5$: $T = 4 \cdot 0.5 = 2.0$ s, against a serial $T_1 = Nd = 20 \cdot 0.5 = 10$ s — a 5× speedup, exactly k. Once the pipeline is full it completes k tasks every d seconds, so the throughput is

$$\lambda \;=\; \frac{k}{d} \quad \text{tasks/sec} \qquad (k = 5,\ d = 0.5 \;\Rightarrow\; \lambda = 10 \text{ tasks/sec}).$$

Now the sanity check that ties it together. **Little's law** says the average number of tasks *in flight*, L, equals throughput times the time each spends in the system:

$$\boxed{L \;=\; \lambda\, d \;=\; \frac{k}{d} \cdot d \;=\; k} \tag{3}$$

The average in-flight count is exactly k — which is the definition of the semaphore. ✓ Plugging $k = 5$, $d = 0.5$: $L = 10 \cdot 0.5 = 5 = k$. The semaphore is not just *a* cap on concurrency; by Little's law it *is* the steady-state in-flight count. That is the cleanest possible statement of what the permit count buys you.

> **Intuition**
>
> Throughput and in-flight count are two readings of the same dial. Crank k and you raise how many tasks overlap ($L = k$) and how fast they drain ($\lambda = k/d$) in lockstep — they are the same k. This is why "add more concurrency" and "more requests in flight at once" are the same lever, and why the provider's rate limit (a cap on λ) and your memory budget (a cap on L) are the same constraint wearing two hats. Both are ceilings on k.

4 Step 3 — the speedup, and where it stops

Speedup is serial time over parallel time:

$$S(k) \;=\; \frac{T_1}{T} \;=\; \frac{Nd}{\lceil N/k \rceil\, d} \;=\; \frac{N}{\lceil N/k \rceil}.$$

When k divides N the ceiling vanishes and $S(k) = k$ — speedup is *linear* in the permit count. But this only holds while $k \le N$: once $k \ge N$ every task is already in its own wave, $W = 1$, and $S = N$ flat. You cannot go faster than "all at once". Tabulating our run, $N = 20$, $d = 0.5$:

k	waves $\lceil N/k \rceil$	wall T	peak concurrency	throughput λ	speedup S
1	20	10.0 s	1	2 /s	1×
2	10	5.0 s	2	4 /s	2×
5	4	2.0 s	5	10 /s	5×
10	2	1.0 s	10	20 /s	10×
20	1	0.5 s	20	40 /s	20×

Sample check, $k = 5$: $W = \lceil 20/5 \rceil = 4$, $T = 4 \cdot 0.5 = 2.0$ s, $\lambda = 5/0.5 = 10$/s, $S = 10/2.0 = 5\times$, peak $= \min(k, N) = 5$. ✓ Each doubling of k (up to N) halves the wall time and doubles both throughput and peak concurrency in lockstep — precisely because all three scale as k.

Watch out

The table makes $k = 20$ look free — 0.5 s, a 20× win — but that column also fires all 20 fetches at the provider simultaneously and holds 20 in-memory results at peak. That is where the real world bites: a provider rate-limited to, say, 8 requests/sec rejects (HTTP 429) the moment $\lambda = k/d$ exceeds it, and 20 large payloads in flight can blow your memory budget. The module gates `FETCH` at `max_concurrency=6` and `MAP` at `12` for exactly this reason: k is chosen as the largest value the provider and machine tolerate, *not* the largest that goes fast. Unbounded concurrency is the bug the semaphore exists to prevent.

5 Step 4 — choosing k: the real tradeoff

Two facts from the table fight each other. Speedup wants k large: $S(k) = k$ while $k \le N$, so every extra permit is another multiple of throughput, for free, until k reaches N and the curve flattens. Stability wants k small: peak concurrency is $\min(k, N)$, and both the provider's rate limit $\lambda_{\max}$ and your memory cap $L_{\max}$ are ceilings on it. The provider limit translates directly into a permit ceiling:

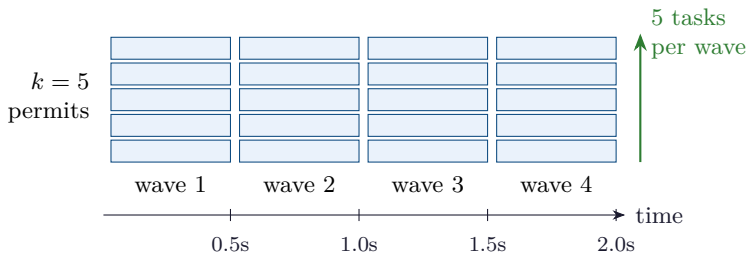
$$\lambda = \frac{k}{d} \le \lambda_{\max} \quad \Longrightarrow \quad k \;\le\; \lambda_{\max}\, d.$$

So the right k is the largest value that respects *both* the rate limit and the memory budget — not the largest that minimises wall time. Past that point, raising k does not speed you up; it trades a small time gain for 429s, retries, and timeouts that make the run *slower* and flakier.

Intuition

There is a sweet spot, and it sits at the binding constraint, not at $k = N$. Below it, you are leaving free speedup on the table — the line is still climbing at slope 1. Above it, you have stopped buying speed and started buying failures. The module's `max_concurrency=12` for `MAP` is an estimate of where that wall is for its provider; tune it down if you see rate-limit errors, up if the provider is generous and the wall time still hurts. The semaphore makes that one number the single, explicit knob for the whole tradeoff.

6 The waves, drawn



Twenty tasks, five permits: four waves of five, each wave $d = 0.5$ s wide, total $T = 2.0$ s. The bars stack $k = 5$ high (the in-flight count, $L = k$) and march $\lceil N/k \rceil = 4$ wide (the waves). Widen the stack (raise k) and the picture gets shorter; the rate limit is a horizontal line the stack must not cross.

7 Concurrency cheat-sheet

Number of waves	$W = \lceil N/k \rceil$
Wall-clock time	$T \approx \lceil N/k \rceil \, d$
Serial baseline	$T_1 = Nd$
Throughput	$\lambda = k/d$
Speedup	$S(k) = N/\lceil N/k \rceil = k$ (while $k \le N$)
In-flight (Little's law)	$L = \lambda d = k$
Peak concurrency	$\min(k, N)$
Rate-limit ceiling on k	$k \le \lambda_{\max} d$
Saturation point	$k = N \Rightarrow W = 1,\ S = N$ (flat after)

8 An honest word about these numbers

The numbers here ($N = 20$, $d = 0.5$ s, exact per task) are deliberately clean so the wave arithmetic lands on whole seconds. Reality is messier: tasks have *varying* durations, so the slowest task in a wave sets that wave's length and the simple Wd becomes an upper bound rather than an exact time; a fast task that finishes early returns its permit, letting a queued task start *before* the wave "ends", which is why the module's real `MAP` over 24 sources at $k = 12$ comes in around 6.8 s rather than a rigid $\lceil 24/12 \rceil \times d$. There is also fixed overhead — connection setup, parsing — that a tiny d would expose. What is *not* illustrative is the shape: wall time falls as $\lceil N/k \rceil$, throughput and in-flight count both rise as k, speedup is linear until k hits N, and a semaphore of size k pins the average concurrency at exactly k by Little's law. Change the constants and the seconds move; the $\lceil N/k \rceil$ shape, and the rate-limit ceiling that stops you raising k, do not.

Appendix A — Reference implementation

Every number in this document is produced by the program below. It computes the wave count, wall time, throughput, peak concurrency, and speedup for each semaphore size, checks Little's law, and prints the module's $N = 24$, $k = 12$ `MAP` wave count for reference. Change `N`, `d`, or the `k` list to model your own pipeline.

```python
# semaphore_concurrency.py -- reproduces every number in "Semaphore-Bounded Concurrency".
import math

N = 20 # number of tasks (e.g. fetches)
d = 0.5 # duration of one task, seconds

def waves(N, k): return math.ceil(N / k) # W = ceil(N/k)
def wall(N, k, d): return waves(N, k) * d # T ~ W*d
def throughput(k, d): return k / d # lambda = k/d
def peak(N, k): return min(k, N) # in-flight cap
def speedup(N, k): return (N * d) / wall(N, k, d) # T1 / T

print(f"serial baseline T1 = N*d = {N*d} s\n")
print(f"{'k':>3} {'waves':>6} {'wall':>7} {'peak':>5} {'thr/s':>6} {'speedup':>8}")
for k in (1, 2, 5, 10, 20):
    print(f"{k:>3} {waves(N,k):>6} {wall(N,k,d):>6.1f}s {peak(N,k):>5} "
```

```
        f"{throughput(k,d):>6.1f} {speedup(N,k):>7.2f}x")

# Little's law: average in-flight L = throughput * d = k
k = 5
L = throughput(k, d) * d
print(f"\nLittle's law (k={k}): L = (k/d)*d = {L:g} -> equals k? {L == k}")

# module MAP stage: N=24 sources, max_concurrency=12
print(f"module MAP: N=24, k=12 -> waves = {math.ceil(24/12)}")

# Expected output:
# serial baseline T1 = N*d = 10.0 s
#
# k waves wall peak thr/s speedup
# 1 20 10.0s 1 2.0 1.00x
# 2 10 5.0s 2 4.0 2.00x
# 5 4 2.0s 5 10.0 5.00x
# 10 2 1.0s 10 20.0 10.00x
# 20 1 0.5s 20 40.0 20.00x
#
# Little's law (k=5): L = (k/d)*d = 5 -> equals k? True
# module MAP: N=24, k=12 -> waves = 2
```

— *end of document* —